

Rapport : Détection de Couleurs et Cercles avec OpenCV

Nathan Vayva

5 novembre 2025

1 Introduction

Ce rapport présente deux projets utilisant OpenCV pour détecter des couleurs spécifiques et identifier des cercles. Le premier projet traite des images fixes, tandis que le second applique la détection sur une séquence vidéo.

2 Traitement d'une Image Fixe

2.1 Lecture et Conversion de l'image

```
1 image = cv.imread("/home/nathan/Documents/mea4/Perception/  
   balle_small.jpg")  
2 hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)
```

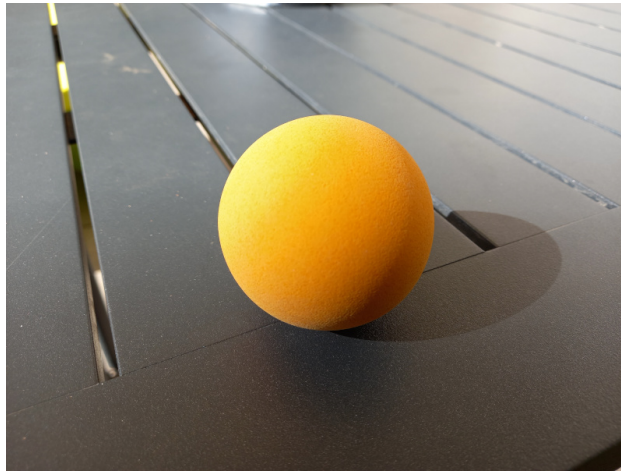


FIGURE 1 – Image originale.

2.2 Sélection de Couleur par Clic

```

1 def get_color(event, x, y, flags, param):
2     global ref_color, ref_color_hsv, passage
3     passage = 0
4     if event == cv.EVENT_LBUTTONDOWN:
5         ref_color = image[y, x].tolist()
6         ref_color_hsv = hsv_image[y, x].tolist()
7         print("Couleur selectionn e (HSV):", ref_color_hsv)

```

2.3 Masque et Nettoyage Morphologique

```

1 lower_bound2 = np.array([max(0, c - e) for c,e in zip(
2     ref_color_hsv, epsilon_hsv)])
3 upper_bound2 = np.array([min(255, c + e) for c,e in zip(
4     ref_color_hsv, epsilon_hsv)])
5 mask2 = cv.inRange(hsv_image, lower_bound2, upper_bound2)
6
7 kernel = np.ones((5,5), np.uint8)
8 mask2_eroded = cv.erode(mask2,kernel, 1)
9 mask2_new = cv.dilate(mask2_eroded,kernel, 1)
10 mask2_dilated = cv.dilate(mask2_new,kernel, 1)
11 mask2_new = cv.erode(mask2_dilated,kernel, 1)

```

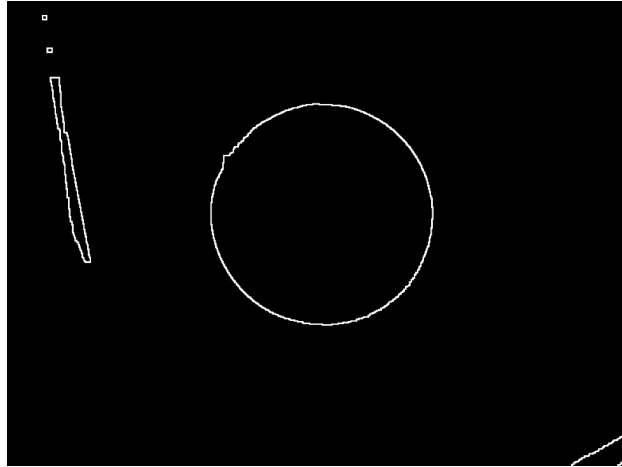


FIGURE 2 – Masque binaire après nettoyage morphologique.

2.4 Détection de Cercles

```

1 kernel_grad = np.ones((3,3))
2 mask2_grad = cv.morphologyEx(mask2_new, cv.MORPH_GRADIENT,
   kernel_grad)
3 circles = cv.HoughCircles(mask2_grad, cv.HOUGH_GRADIENT, dp
   =1, minDist=1,
4                               param1=10, param2=30, minRadius=10,
                               maxRadius=1000)

```

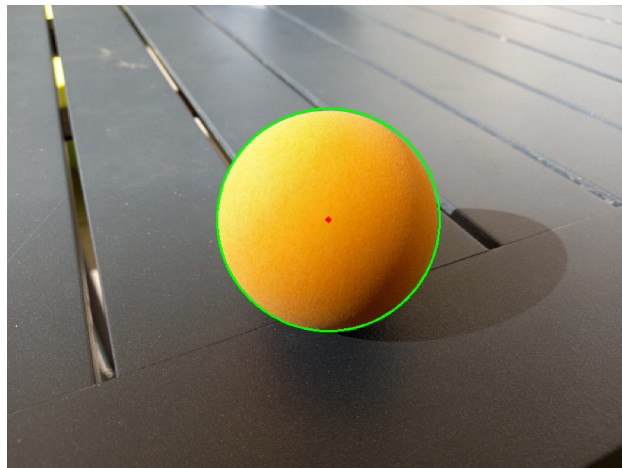


FIGURE 3 – Cercle détecté sur l'image.

3 Traitement d'une Séquence Vidéo

3.1 Lecture et Initialisation

La vidéo est ouverte avec `cv.VideoCapture`. La première image est analysée pour déterminer une couleur candidate pour la balle.

```
1 cap = cv.VideoCapture("/home/nathan/Documents/mea4/Perception  
  /balle.mp4")  
2 ret, image = cap.read()  
3 hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)  
4 epsilon_hsv = [10, 120, 120]
```

3.2 Balayage Initial pour Couleur de Référence

Un balayage rapide de l'image détecte un pixel dont la couleur correspond à la balle, et initialise `ref_color_hsv`.

```
1 while i <= longueur and j <= largeur - 5 and not 'reussite'  
  in locals():  
2     if 15 < hsv_image[i, j, 0] < 20 and 150 < hsv_image[i, j,  
      1] < 255 and 150 < hsv_image[i, j, 2] < 230:  
3         ref_color_hsv = hsv_image[i, j, :]  
4         reussite = True
```

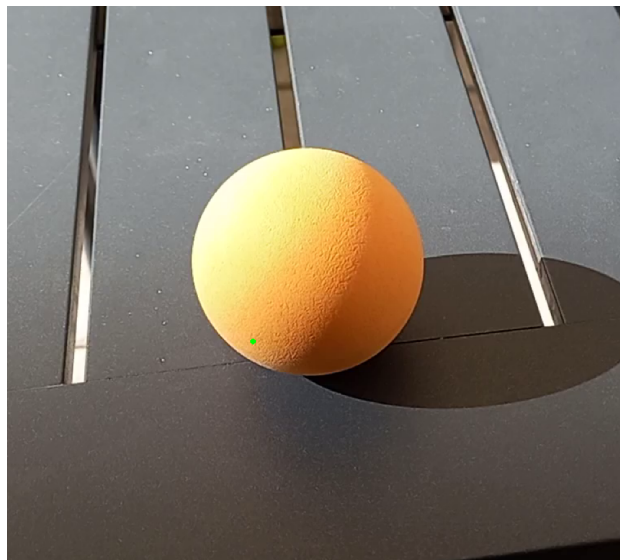


FIGURE 4 – Première image de la vidéo avec point vert indiquant la couleur détectée.

3.3 Traitement Image par Image

Chaque frame est convertie en HSV, un masque est calculé et nettoyé morphologiquement, puis la détection de cercles est appliquée.

```
1 while cap.isOpened():
2     ret, image = cap.read()
3     if not ret:
4         break
5     hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)
6     lower_bound2 = np.array([max(0, c - e) for c, e in zip(
7         ref_color_hsv, epsilon_hsv)])
8     upper_bound2 = np.array([min(255, c + e) for c, e in zip(
9         ref_color_hsv, epsilon_hsv)])
10    mask2 = cv.inRange(hsv_image, lower_bound2, upper_bound2)
11
12    kernel = np.ones((5,5), np.uint8)
13    mask2_eroded = cv.erode(mask2, kernel, 1)
14    mask2_new = cv.dilate(mask2_eroded, kernel, 1)
15    mask2_dilated = cv.dilate(mask2_new, kernel, 1)
16    mask2_new = cv.erode(mask2_dilated, kernel, 1)
17
18    kernel_grad = np.ones((3,3), np.uint8)
19    mask2_grad = cv.morphologyEx(mask2_new, cv.MORPH_GRADIENT,
20        kernel_grad)
21
22    circles = cv.HoughCircles(mask2_grad, cv.HOUGH_GRADIENT,
23        dp=1, minDist=1,
24        param1=10, param2=30, minRadius
25        =10, maxRadius=1000)
```

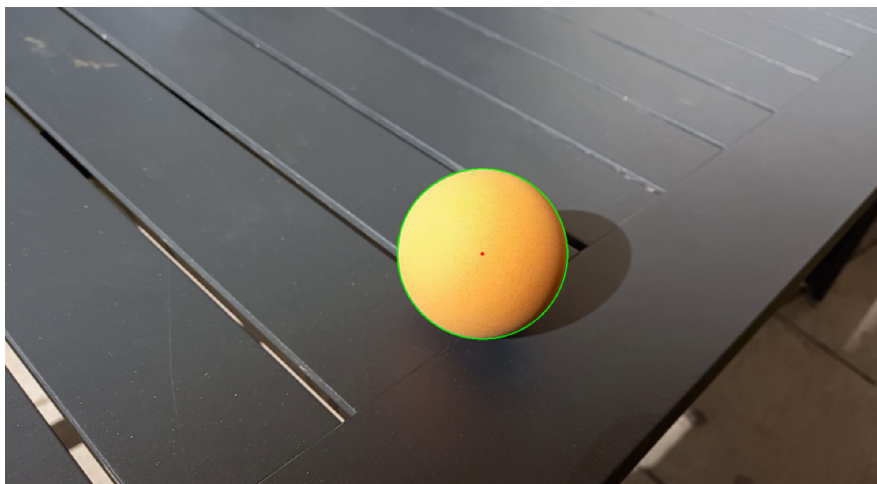


FIGURE 5 – Exemple de frame annotée avec cercle détecté.

4 Conclusion

Les deux codes démontrent la puissance d'OpenCV pour :

- Détecter une couleur dans une image ou une vidéo.
- Nettoyer le masque via des opérations morphologiques.
- Détecter des cercles avec la transformée de Hough.
- Appliquer ces traitements en temps réel pour les vidéos.

Ces techniques sont fondamentales pour le suivi d'objets et la vision par ordinateur.